

1. Parametric vs. Non-Parametric Models

Item	Class	Justification
(a) Neural network dynamics model	Parametric	A fixed architecture with a fixed-size parameter vector represents the transition function; after training, predictions come from the learned weights rather than stored samples.
(b) Replay buffer as empirical transition model	Non-parametric	The buffer itself is the model. Prediction is based on sampled/matched stored transitions, so memory and complexity grow with data.
(c) Symbolic PDDL-style planning model	Parametric	Assuming the symbolic domain/schema is fixed, a PDDL model is a compact set of predicates, action schemas, preconditions, and effects; its size does not grow with observations.
(d) Gaussian process dynamics model	Non-parametric	Even with kernel hyperparameters, predictions depend on the stored training points/kernel matrix, so model complexity grows with the dataset.
(e) Table of all observed gridworld transitions	Non-parametric	As phrased, it stores empirical transitions/counts directly, so the representation scales with observed state-action-next-state entries. A pre-specified finite transition-probability table could be viewed as tabular parametric, but an observed-transition table is instance-based.

2. Model Predictive Control

(a) MPC procedure. At each step, observe the current state, use a dynamics model to roll out candidate action sequences over a finite horizon, evaluate their predicted cumulative reward/cost, choose the best sequence, execute only its first action, observe the next real state, shift the horizon forward, and replan. This is receding-horizon control.

(b) Why only the first action? Later planned actions depend on predicted future states, which may be wrong due to model error, disturbances, or stochasticity. Executing one action and replanning from the newly observed true state turns open-loop planning into feedback control and reduces compounding error.

(c) Example. Autonomous driving: the car repeatedly plans a short trajectory respecting lanes, nearby vehicles, speed, and comfort, executes the immediate steering/throttle/brake command, then replans as traffic changes.

(d) Vanilla MPC vs. TD-MPC. Vanilla MPC optimizes a short finite-horizon trajectory with a dynamics model and short-term rewards/costs, often using an undiscounted objective, so it can be myopic. TD-MPC keeps receding-horizon planning but connects it to TD learning: it plans locally while a discounted terminal value estimates long-horizon return.

TD-MPC's enhancements include: **(1) task-oriented latent dynamics**, planning in compact reward-relevant latent space instead of raw state/pixel space; **(2) learned terminal Q-value**, which estimates return beyond the short planning horizon; **(3) joint TOLD learning**, training the encoder, latent dynamics, reward predictor, Q-function, and policy together with reward, TD-value, and consistency losses; **(4) latent consistency loss**, matching predicted latent states to target encoded next observations without reconstructing pixels; **(5) policy-guided trajectory optimization**, where a learned policy proposes extra candidate trajectories for MPPI planning; **(6) planning-based exploration and horizon scheduling**, using a lower bound on sampling variance and gradually increasing horizon while the model is immature; and **(7) warm-started receding-horizon planning**, reusing the shifted previous mean action sequence to reduce optimization cost. Therefore, TD-MPC is a hybrid model-based/model-free method: short-term rewards come from learned latent rollouts, while long-term behavior is shaped by TD-trained value learning.